Proof of Concept: Hall Booking Management

Project Name: VenueHub

Developer: Team 3

Tech Stack: MERN - React, Node.js, Express.js, MongoDB

1. Introduction & Problem Statement

Overview of Hall Booking System

VenueHub is a centralized digital platform designed to help users browse, filter, and reserve event spaces for weddings, corporate seminars, and parties. The system provides a "General Hall Booking" interface that simplifies the discovery of venues based on location, capacity, and pricing.

The Need for Automation

Traditional manual booking systems are inefficient and prone to errors. By moving to an automated system, venue owners can spend less time managing paperwork and more time hosting events.

Problems in Manual Systems

- **Time Conflicts:** Difficulty in tracking exact start and end times across multiple paper logs.
- **No Proper Tracking:** Lack of a central database makes it hard to retrieve past booking history or customer details.
- **Manual Effort:** Staff must manually check availability, which is slow and frustrating for the customer.

2. System Requirements

Functional Requirements

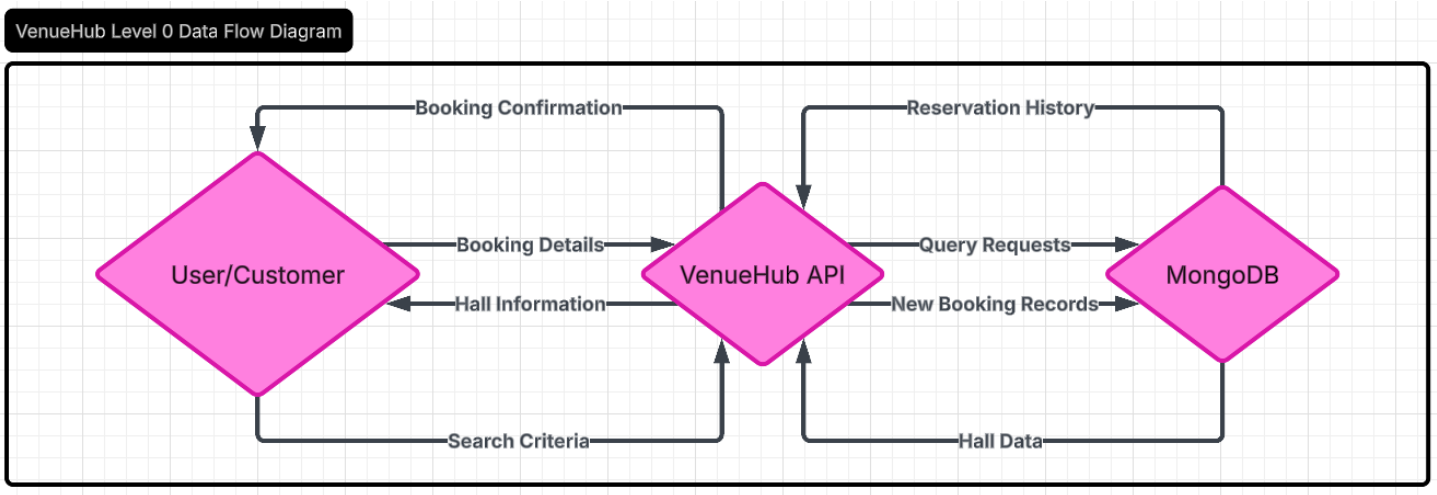
- **Venue Discovery:** Users must be able to view a gallery of halls with images, ratings, and key specs (Capacity/Parking).
- **Booking Submission:** The system must allow users to submit their name, event date, and time slot via a digital form.
- **Data Persistence:** All booking data must be saved in a cloud-based database to ensure the information is permanent and accessible.

3. System Architecture & Design

Overall Architecture

The project utilizes the **MERN** stack for a seamless full-stack experience:

- **Frontend (React):** A modern, responsive UI used for browsing and interacting with hall data.
- **Backend (Node.js & Express):** A server-side environment that processes booking requests and communicates with the database.
- **Database (MongoDB):** A NoSQL database used to store venue information and booking records.



4. Database Design

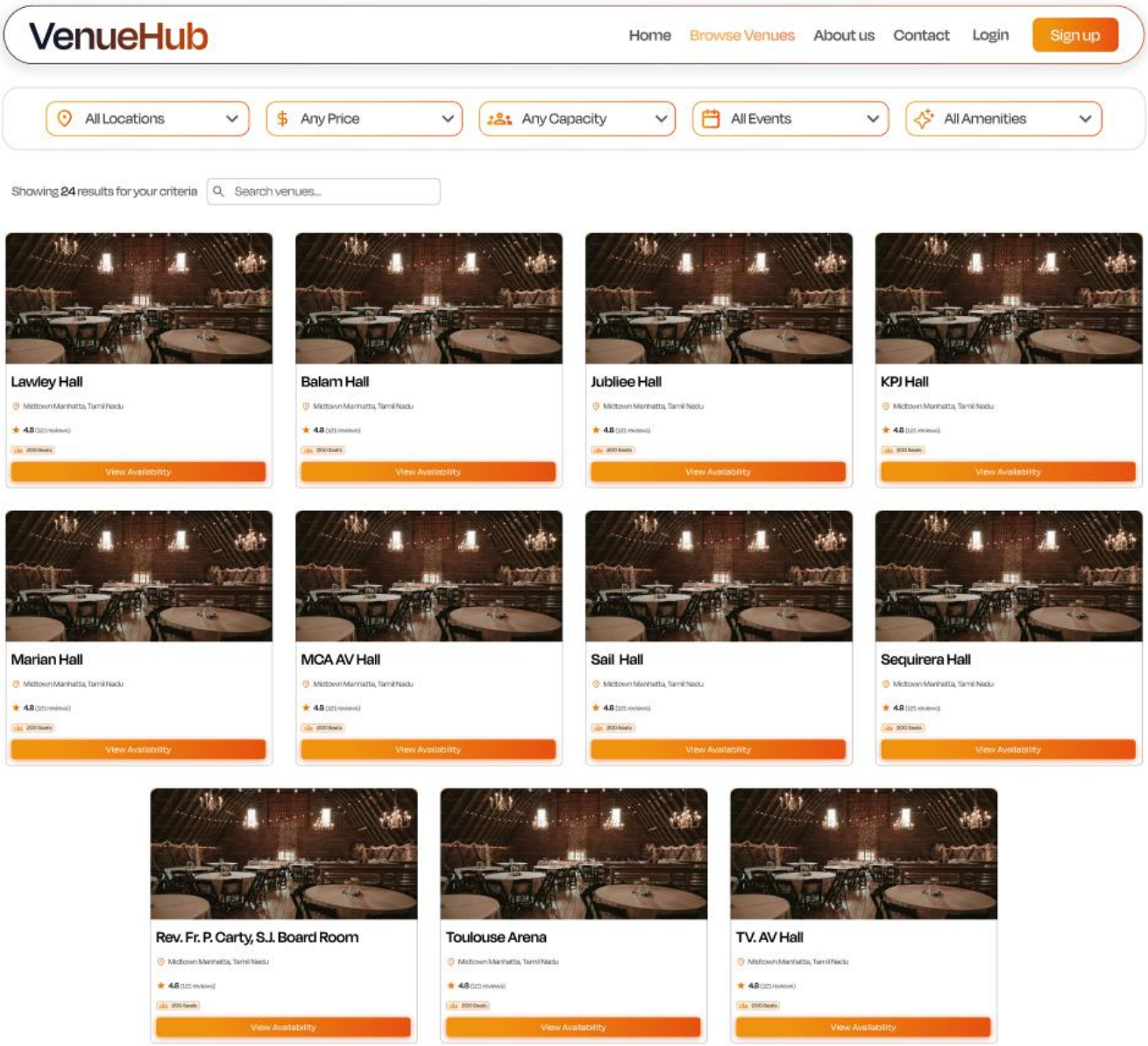
Collection: `Bookings`

Field Name	Data Type	Description
`hallName`	String	The name of the reserved venue (e.g., Skyline Executive Suite).
`customerName`	String	The name of the person making the reservation.
`date`	String	The scheduled date for the event.
`timeSlot`	String	The specific duration requested (e.g., 10:00 AM - 2:00 PM).

5. Core Functionality

The Booking Process

1. **User Selection:** The user navigates through the React frontend and clicks "View Availability" on a hall card.
2. **Input Collection:** A modal prompt appears, requesting the user's contact name and event timing.
3. **API Transmission:** The frontend sends a POST request to the Node.js backend with the JSON data.
4. **Final Storage:** The backend receives the data and saves it directly into the MongoDB Atlas cloud.



6. Features

Current Features

- **Modern Marketplace UI:** A professional layout featuring venue cards with verified badges and ratings.
- **Cloud Integration:** Real-time data storage using MongoDB, ensuring no data loss.

7. Setup & Installation

7.1 Prerequisites

- **Node.js & npm:** Required for the runtime environment and package management.
- **Visual Studio Code:** The recommended IDE for code modification.
- **MongoDB Atlas Account:** To host the cloud database.

7.2 Backend Setup (Node.js & Express)

- Navigate to the `/server` directory in your terminal.
- Install dependencies: `npm install express mongoose cors dotenv`.
- Create a `.env` file and add your MongoDB connection string: `MONGO_URI=your_mongodb_link`.
- Start the server: `node server.js` or `npm start`.

7.3 Frontend Setup (React)

1. Navigate to the `/client` directory (where your **v0.dev** code is located).
2. Install dependencies: `npm install`.
3. Start the React application: `npm start`.
4. The system will automatically open in your browser at http://localhost:3000.